

AI Research Assistant - Funcionalidades para portfolio

Documento de demo para presentar y probar el proyecto AI Research Assistant en un portfolio técnico.

Resumen ejecutivo

AI Research Assistant es una aplicación full-stack para consultar documentos PDF mediante RAG. Permite subir PDFs, extraer texto, indexar fragmentos en ChromaDB, hacer búsquedas semánticas, conversar con un asistente con memoria y mostrar fuentes por documento y página. También incluye un modo agente tipo LangGraph, streaming SSE, extracción de facturas a JSON, guardrails, métricas locales, evaluación RAG, servidor MCP, tests y Docker.

Stack principal

- Backend: FastAPI, PyMuPDF, ChromaDB, Groq opcional y fallback local.
- Frontend: React, TypeScript y Vite.
- IA/RAG: chunking, embeddings configurables, búsqueda semántica, grounding con fuentes y control de presupuesto de contexto.
- Persistencia local: PDFs subidos, ChromaDB, historial JSON y métricas SQLite.
- Entrega: Dockerfiles de backend/frontend y docker-compose.

Funcionalidades principales para probar

1. Subida e indexación de PDFs

- Subida de archivos PDF desde la interfaz web.
- Validación de extensión, tipo MIME y tamaño máximo.
- Nombre seguro para almacenar el documento.
- Extracción de texto con PyMuPDF.
- Fallback OCR para PDFs escaneados cuando está disponible.
- Respuesta con páginas, caracteres, método de extracción, preview de texto, document_id, colección y número de chunks indexados.

2. RAG sobre documentos

- División del texto en fragmentos con metadatos.
- Embeddings locales por defecto con HashingVectorizer para funcionar sin API paga.
- Opción de Sentence Transformers para mejorar recuperación semántica.
- Persistencia en ChromaDB.
- Búsqueda semántica de los fragmentos más relevantes.
- Respuestas en español con fuentes recuperadas.

3. Chat clásico con memoria

- Endpoint `/chat` para preguntar sobre PDFs indexados.
- Historial por `session_id`.
- Recuperación de conversaciones recientes para preguntas de seguimiento.
- Respuesta con modelo usado, indicador de LLM/fallback, fuentes y tokens estimados.

- Gestión de conversaciones desde el frontend.

4. Agente IA tipo LangGraph

- Endpoint `/agent/chat` con flujo agentico visible.
- Pasos demostrables: pensar, decidir, usar herramienta, evaluar contexto y responder.
- Uso de la búsqueda semántica como herramienta.
- Devuelve `agent_steps`, framework, fuentes, historial, tokens estimados y contextos incluidos.
- Ideal para explicar arquitectura agentica en entrevistas.

5. Streaming SSE

- Endpoint `/chat/stream`.
- Emisión de metadata, tokens y evento final `done`.
- UI con modo “Streaming SSE” para mostrar la respuesta progresivamente.

6. Búsqueda de chunks

- Endpoint `/search` para inspeccionar la recuperación antes de generar una respuesta.
- Control del límite de resultados.
- Visualización de archivo, página, distancia de similitud y texto recuperado.

7. Extracción de facturas PDF a JSON

- Endpoint `/extract-invoice`.
- Extrae cliente, importe, moneda, fecha y número de factura.
- Incluye confianza, campos faltantes, preview y método de extracción.
- Útil como demo adicional de document understanding.

8. Guardrails y seguridad básica

- Inspección de la pregunta para detectar prompt injection.
- Validación de fuentes requeridas en respuestas RAG.
- Rechazo de preguntas sospechosas con error estructurado.

9. Evaluación RAG

- Endpoint `/evaluation/rag`.
- Métricas determinísticas: faithfulness, context precision, recall proxy, riesgo de alucinación, grounding y cobertura de fuentes.
- Botón en frontend para evaluar la última respuesta.

10. Observabilidad local

- Endpoint `/metrics`.
- Métricas de eventos, latencia media, tokens estimados y fuentes promedio.
- Registro local para medir interacciones sin servicios externos.

11. Cobertura AI Engineer

- Endpoint `/ai-topics` con capacidades cubiertas.
- Lista temas como RAG, chunking, embeddings, semantic search, citations, MCP, LangChain adapters, agentes, guardrails, token optimization, streaming, métricas y OCR.
- Explica explícitamente qué APIs pagas se excluyeron por coste.

12. Servidor MCP

- Servidor MCP por stdio ejecutable con `python -m app.mcp_server`.
- Herramientas: listar documentos, buscar base PDF, preguntar al asistente RAG y preguntar al agente.
- Permite conectar la base RAG con clientes compatibles con Model Context Protocol.

13. Frontend de demo

- Hero con descripción del producto.
- Panel de subida e indexación de PDFs.
- Panel de extracción de facturas.
- Panel de búsqueda de chunks.
- Panel de cobertura AI Engineer y métricas.
- Panel de historial de sesiones.
- Chat con tres modos: Agente IA, RAG clásico y Streaming SSE.
- Visualización de fuentes y pasos del agente.

14. Docker y ejecución local

- Dockerfile para backend.
- Dockerfile para frontend con Nginx.
- `docker-compose.yml` para levantar ambos servicios.
- Variables de entorno para API, CORS, embeddings, ChromaDB, historial, uploads, Groq y presupuesto de tokens.

15. Tests y checks

- Tests de backend con pytest.
- Build de frontend con TypeScript y Vite.
- Casos cubiertos: subida PDF, historial, agente, MCP, RAG con LangChain y presupuesto de contexto.

Flujo recomendado de demo en portfolio

1. Levantar backend y frontend.
2. Subir un paper o documentación técnica en PDF.
3. Mostrar páginas, caracteres, método de extracción y chunks indexados.
4. Buscar chunks con una pregunta concreta para enseñar recuperación semántica.
5. Preguntar al asistente en modo Agente IA.
6. Abrir “Pasos del agente” para demostrar razonamiento orquestado y uso de herramientas.
7. Abrir “Fuentes recuperadas” para enseñar grounding por archivo y página.
8. Hacer una pregunta de seguimiento para demostrar memoria por sesión.
9. Probar modo Streaming SSE para enseñar respuesta progresiva.

10. Evaluar la última respuesta RAG y mostrar riesgo de alucinación y métricas.
11. Subir una factura PDF y enseñar el JSON extraído.
12. Explicar Docker, MCP y próximos pasos de deploy.

Endpoints clave

- GET /health: estado del backend.
- GET /metrics: métricas locales.
- GET /ai-topics: capacidades AI Engineer cubiertas.
- POST /upload-pdf: subir, extraer e indexar PDF.
- POST /extract-invoice: extraer factura a JSON.
- GET /search: búsqueda semántica de chunks.
- POST /chat: chat RAG clásico.
- POST /chat/stream: chat con streaming SSE.
- POST /agent/chat: chat con agente IA.
- GET /chat/sessions: listar conversaciones.
- GET /chat/sessions/{session_id}: recuperar historial.
- POST /evaluation/rag: evaluar una respuesta RAG.